# Edge cases (when the answer spans two chunks) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.


- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing


## Hint 1 — The idea, and the exact pieces

The exercise has three moving parts: fixed-size chunking, embedding + cosine-similarity search (the same search you built in `embedding_similarity`), and the `k` parameter that controls how many chunks a search returns.

Fixed-size chunking with plain slicing looks like this:

```
# chunking_practice.py — Edge cases section
for i in range(0, len(text), chunk_size):
    chunk = text[i:i + chunk_size]
```

This has no idea where a sentence ends — it just cuts every `chunk_size` characters. That's exactly the property you want here: pick a `chunk_size` deliberately small enough, relative to one long sentence, that the cut lands in the middle of the fact you care about.

The exact pieces:

- One long test sentence containing the whole fact.
- `chunk_by_chars(text, chunk_size)` sized so it produces exactly 2 chunks, with the cut in the middle of the fact.
- Reuse `get_embedding` and `cosine_similarity` unchanged.
- A `search(question, chunks, chunk_embeddings, k)` function, so you can compare `k=1` against `k=len(chunks)` directly.

A bigger `k` isn't actually a fix — it's a workaround. Even at `k=len(chunks)` (every chunk, guaranteed to include both halves), you only get lucky because this is a 2-chunk toy example. In a

real knowledge base with thousands of chunks, a bigger `k` just makes it *more likely* the missing half scores well enough to make the cut — it still depends on that chunk actually ranking high enough, and a bigger `k` also means more irrelevant chunks get handed to the model as context (this document's Core Concepts calls this "lost in the middle").

The real fix is **chunking with overlap**: instead of cutting the document into non-overlapping blocks, each new chunk starts a little *before* where the previous one ended, so a fact sitting near a boundary ends up whole inside at least one chunk — not permanently split in two, regardless of `k`. The rule: any text up to `overlap` characters long always fits whole in some chunk. A fact longer than `chunk_size` can never fit, whatever the overlap — so check the fact's length first.

```
# chunking_practice.py — Edge cases section
def chunk_with_overlap(text: str, chunk_size: int, overlap: int) -> list[str]:
    chunks = []
    # smaller than chunk_size -- chunks now overlap
    step = chunk_size - overlap
    i = 0
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
        i += step
    return chunks
```

Build the *same* boundary-splitting test document from Hint 2 below, chunk it with overlap instead of the plain non-overlapping slicer, and check: does the fact now appear whole inside at least one chunk, so even `k=1` can return the complete answer?

## Hint 2 — The plan, and almost the whole thing

```
# chunking_practice.py — Edge cases section
DOCUMENT: str = (
    "The quarterly meeting will be held at 3 PM on Friday in Room 204, "
    "and everyone from the product and engineering teams is expected "
    "to attend without exception."
)
CHUNK_SIZE: int = 80

def chunk_text(text: str, chunk_size: int) -> list[str]:
    chunks = []
    for i in range(0, len(text), chunk_size):
        chunks.append(text[i:i + chunk_size])
    return chunks
```

What's missing: a `search(question, chunks, chunk_embeddings, k)` function that scores and sorts, plus a `main()` that runs `k=1` and `k=len(chunks)` and compares. Write that yourself, then compare against the Solution.

Once that's working, add the real fix — overlap:

```
document = same quarterly-meeting sentence as before
the fact, "3 PM" ... "attend", is 101 characters long
rule: text up to `overlap` characters long always fits whole in a chunk
so: chunk_size = 140, overlap = 105   (80 is smaller than the fact itself)

chunk_with_overlap(document, chunk_size, overlap) -> should produce
    chunks where "3 PM" and "attend" sit inside the same chunk

embed each overlapping chunk

search with k=1 against the overlapping chunks:
    does the single top chunk now contain the whole fact,
    where the non-overlapping version at k=1 didn't?
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# chunking_practice.py — Edge cases section
def chunk_with_overlap(text: str, chunk_size: int, overlap: int) -> list[str]:
    chunks: list[str] = []
    step = chunk_size - overlap
    i = 0
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
        # your turn: move i forward by `step`, not chunk_size
        ...
    return chunks

# your turn: build overlap_chunks from DOCUMENT using chunk_with_overlap,
# embed them, then run search(QUESTION, overlap_chunks, ..., k=1) and compare
# the result against the non-overlapping version's k=1 result
```

Finish `chunk_with_overlap` and the `k=1` comparison yourself, then compare all of your finished versions against the Solution.

Full solution: Show me the solution